Evaluating Hybrid KEM/DSA for KpqC and NIST PQC on ARM Cortex-M4

Minjoo Sim[1][0000-0001-5242-214X], Minwoo Lee[1][0000-0002-2356-3055], Subeen Cho[2][0009-0004-4352-8021], Yulim Hyoung[2][0009-0009-7530-6497], and Hwajeong Seo[2][0000-0003-0069-9061]

[1] Department of Information Computer Engineering, Hansung University, Seoul (02876), South Korea
{minjoos9797, minunejip}@gmail.com
[2] Department of Convergence Security, Hansung University, Seoul (02876), South Korea
{chosubin1208,yulim4hyoung, hwajeong84 }@gmail.com

Abstract. Primitive-only PQC benchmarks are insufficient for attributing composed hybrid costs on Cortex-M4 because shared hash backends, signing-time variability, and fixed classical/wrapper work affect measured performance. We implement a common bare-metal Cortex-M4 harness for representative KpqC/NIST families, measuring uniform Hash-CT hybrid KEM benchmark rows with X25519 and Bindel et al. hybrid-signature AND-combiner rows. The goal is composed-cost attribution under a uniform benchmark transcript rather than primitive-only ranking.

Our measurements show three effects relevant to cost attribution. First, replacing only Keccak-$f[1600]$ changes SHAKE-heavy signing by up to 2.16×, while SPHINCS$^+$-SHA2 and FN-DSA control rows remain at 1.00×. Second, median-only signature tables do not characterize the observed signing-time variability. In the nominal level-5 signing rows, HAETAE5 has lower median and mean signing latency than FN-DSA-1024, but its largest observed $N = 100$ measurement reaches 4.53× its median while the FN-DSA-1024 maximum is less than 1% above its median. These maxima are descriptive and do not estimate tail probabilities. Third, a local SMAUG-T backend improves standalone SMAUG-T by 1.79–1.82×, but the composition-level speedup drops to 1.42–1.65× with SMAUG-T+X25519. The KEM rows report 3.7–8.8 M cycles of summed KeyGen/Encaps/Decaps work for lattice-based KEMs and 19.6–74.5 M cycles for HQC. Together, the results motivate reporting composed-cost attribution, backend provenance, and observed variability alongside primitive timings.

Keywords: Post-Quantum Cryptography · Hybrid KEM · Hybrid DSA · ARM Cortex-M4 · Embedded Benchmarking · KpqC

1 Introduction

Post-quantum deployment is now an integration problem, not only a primitive-selection problem. On microcontrollers, the combiner, transcript, byte layout, classical partner, and shared low-level backend can materially affect latency, wire size, stack, and flash [13,28]. This paper shows that Cortex-M4 hybrid post-quantum cryptography (PQC) cannot be understood from primitive timings alone, because shared symmetric backends, signing-time variability, and optimization dilution can change composed-cost conclusions even when the component primitives are fixed.

We evaluate representative families selected by the National Institute of Standards and Technology (NIST) and the Korean Post-Quantum Cryptography (KpqC) competition. NIST has finalized ML-KEM, ML-DSA, and SLH-DSA [24,25,26], while FN-DSA and HQC have been selected for ongoing NIST standardization [29,27]. KpqC has selected SMAUG-T, NTRU+, HAETAE, and AIMer [18]. These families cover lattice-based, code-based, multiparty computation-in-the-head, and hash-based designs, making them a useful portfolio for studying composed cost, backend sensitivity, and signing-time variability.

The key encapsulation mechanism (KEM) rows use a uniform ciphertext-hashing (Hash-CT) benchmark transcript with X25519, fixed encodings, labels, selected KpqC/NIST KEM components, and the Quantum Superiority Fighter (QSF) component interface. Section 2.1 defines the exact measurement-interface scope and X-Wing/QSF background. The digital signature algorithm (DSA) rows implement Bindel et al.'s hybrid signature AND-combiner with label-bound component signatures [4]. Unlike prior embedded hybrid-PQC studies that mainly target Transport Layer Security (TLS)/X.509 stacks [6,15,32], this paper measures the primitive/harness layer and focuses on composed-cost attribution under one bare-metal Cortex-M4 harness.

Contributions.

1. **Composed-cost attribution for hybrid PQC.** We implement a common bare-metal Cortex-M4 harness for uniform Hash-CT benchmark rows and Bindel et al. hybrid signature AND-combiner rows, fixing byte layouts, labels, interfaces, backends, and target configuration so that classical-leg cost, wrapper work, and shared backend effects are visible.
2. **Backend sensitivity and observed signing-time variability.** We show that a Keccak-$f[1600]$ backend swap changes SHAKE-heavy rows while control rows remain unchanged, and report the observed variability of rejection-sampling signatures alongside their medians. We also report cycles, wire sizes, peak stack, code estimates, and standalone checks. These experiments are implementation-sensitivity analyses and are not claimed as phenomena unique to hybrid composition.
3. **SMAUG-T backend and optimization dilution.** We contribute a local Cortex-M4 SMAUG-T arithmetic backend and use it as a case study show-

ing that standalone speedups shrink within SMAUG-T+X25519 compositions because fixed classical, hashing, and wrapper work remain.

Section 2 defines the measured constructions and security scope. Section 3 describes the implementation and measurement setup. Section 4 presents the measurements and backend case studies, and Section 5 concludes.

2 Preliminaries and Related Work

2.1 Hash-CT QSF Benchmark Rows

Barbosa et al. proposed X-Wing, an ML-KEM-768/X25519 KEM built from the QSF combiner [2]. The Internet Engineering Task Force (IETF) draft specifies ML-KEM-768/X25519 X-Wing [12], and the X-Wing KEM team publishes a C implementation [34]. The X-Wing paper also studies a Hash-CT transcript that includes the post-quantum ciphertext. We use Hash-CT QSF benchmark rows only as a local measurement label for the uniform transcript of Equation (1), not as a new KEM construction, a new combiner, or a measurement of the IETF X-Wing transcript. Under one benchmark interface, the measured rows hash the post-quantum ciphertext together with the component shared secrets and classical transcript fields for ML-KEM, SMAUG-T, NTRU+, and HQC. In particular, the IETF ML-KEM-768/X25519 X-Wing transcript omits the ML-KEM ciphertext from the final combiner hash, whereas our Hash-CT rows include ct_{pq}. For large-ciphertext KEMs such as HQC, this adds an additional Hash-CT hashing cost relative to a no-ct_{pq} transcript. The DSA rows are separate and implement Bindel et al.'s hybrid signature AND-combiner as treated in Section 2.2.

For a selected algorithm pair, public keys, secret keys, and ciphertexts are fixed concatenations of the post-quantum and classical byte strings. The measured Hash-CT transcript computes the final shared secret as

$$ss = \mathsf{SHA3\text{-}256}(label \parallel ss_{\mathrm{pq}} \parallel ss_{\mathrm{cl}} \parallel ct_{\mathrm{pq}} \parallel ct_{\mathrm{cl}} \parallel pk_{\mathrm{cl}}), \tag{1}$$

where ss_{pq} and ss_{cl} are component shared secrets, ct_{pq} and ct_{cl} are component encapsulation tokens, and pk_{cl} is the long-term classical public key. The label has a fixed prefix and both component names, and fixed field lengths make parsing injective within each pair. The post-quantum public key is not hashed as a separate field, so adding it would define a different public-key-bound transcript.

Figure 1(a) shows the IETF X-Wing key-generation background. For the reported KeyGen measurements, the harness instead invokes the post-quantum and X25519 key-generation routines independently. Figure 1(b) shows the Hash-CT sender encapsulation measured in this work. During decapsulation, the receiver recovers both component shared secrets and reapplies Equation (1).

Security scope. The tables quantify costs for the Hash-CT benchmark rows above. The cited X-Wing/QSF work provides construction background for the component flow and interface. This paper does not derive fresh reductions for every substituted KpqC/NIST KEM component or claim side-channel resistance, fault resistance, or formal constant-time guarantees.

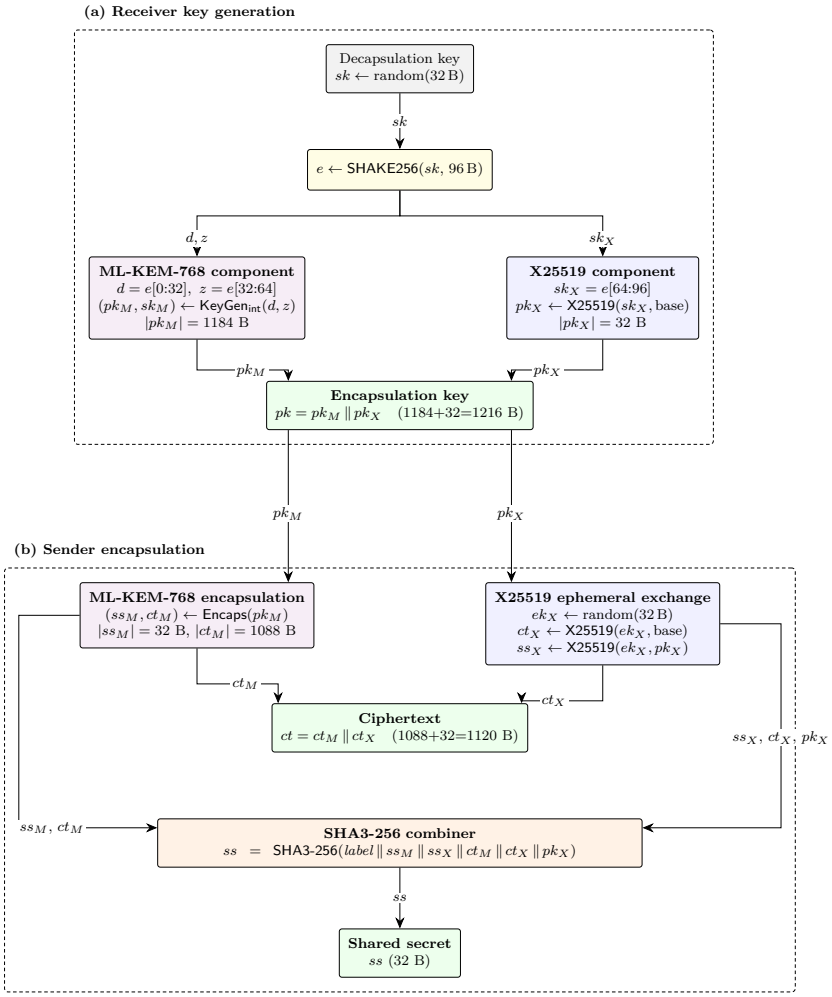
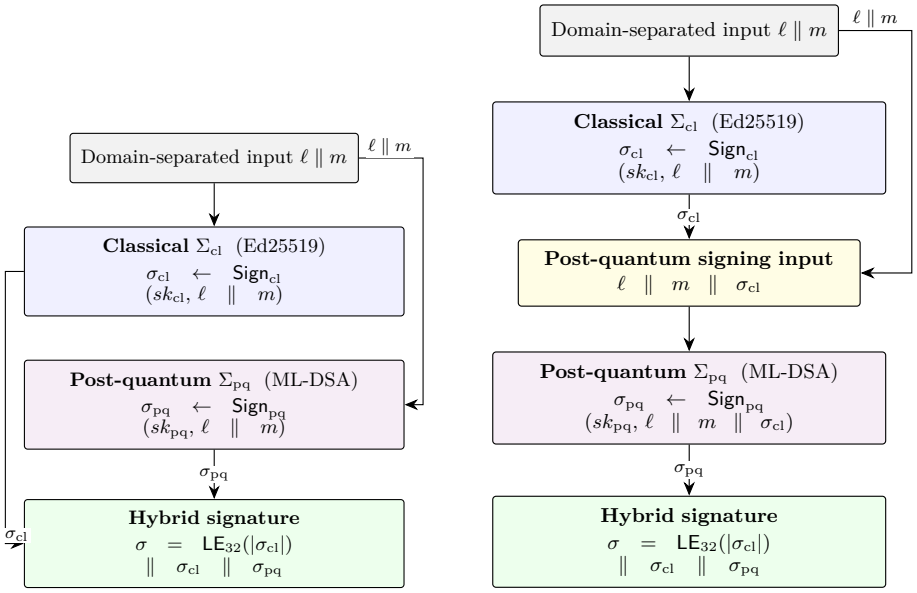


Fig. 1: X-Wing receiver key generation and Hash-CT sender encapsulation for ML-KEM-768/X25519.



(a) Concat mode: both components sign $\ell \parallel m$.

(b) Strong-nesting mode: the post-quantum component signs $\ell \parallel m \parallel \sigma_{\text{cl}}$.

Fig. 2: Concat and strong-nesting hybrid-signature signing flows implemented in the harness.

2.2 Hybrid Signature Combiner

For signatures, we implement the AND-combiner of Bindel et al. [4], where the classical and post-quantum schemes sign label-bound inputs and verification accepts only if both component signatures verify. Let Σ_{cl} and Σ_{pq} denote the classical and post-quantum schemes. With fixed label ℓ, the primary concat mode is

$$\sigma_{\text{cl}} \leftarrow \mathsf{Sign}_{\text{cl}}(sk_{\text{cl}}, \ell \parallel m), \qquad \sigma_{\text{pq}} \leftarrow \mathsf{Sign}_{\text{pq}}(sk_{\text{pq}}, \ell \parallel m). \tag{2}$$

The hybrid signature is serialized with a 32-bit little-endian length prefix, denoted LE_{32}, as $\sigma = \mathsf{LE}_{32}(|\sigma_{\text{cl}}|) \parallel \sigma_{\text{cl}} \parallel \sigma_{\text{pq}}$. The length prefix makes parsing unambiguous. A strong-nesting mode is retained only for overhead checks. In that mode, the post-quantum input is $\ell \parallel m \parallel \sigma_{\text{cl}}$. If concat and nested modes are used in one protocol context, labels must be mode-specific.

2.3 Related Work

Related work falls into three groups. X-Wing, Starfighters, Request for Comments (RFC) 9794, NIST transition guidance, and Bindel et al. provide the hybrid-construction background [2,12,11,13,28,4]. Protocol-level studies measure Transport Layer Security (TLS)/X.509 PQC or hybrid costs, including TLS

based on ring learning with errors, Kyber/SPHINCS+ in mbed TLS, KEMTLS versus PQ-TLS, and KpqC in TLS/OpenSSL [5,6,15,32]. The closest KpqC-over-TLS study targets Cortex-A76 with OpenSSL, whereas we measure the bare-metal Cortex-M4 primitive/harness layer. Finally, pqm4, PQClean, and scheme-specific M4 work provide the implementation lineage [16,31,7,19,10,23]. We fix the KEM wrapper, classical leg, transcript layout, footprint accounting, and shared backend without adding a TLS stack. This makes backend sensitivity, observed signing-time variability, and optimization dilution visible below the protocol layer.

Ghinea et al. [14] evaluated an Elliptic Curve Digital Signature Algorithm (ECDSA)–Dilithium hybrid on an nRF52840 Cortex-M4F security key, including signing-time distributions and Client to Authenticator Protocol timeouts, thereby documenting rejection-sampling variability in an embedded hybrid setting. We do not claim this variability as a new observation. Our evaluation instead provides broader KpqC/NIST coverage under a common bare-metal harness with fixed interfaces, backends, classical partners, and target configuration.

3 Implementation and Experimental Setup

3.1 Implementation Design

The harness separates Hash-CT KEM orchestration from Bindel et al. hybrid signature orchestration. KEM wrappers call component key generation, encapsulation, and decapsulation, concatenate post-quantum (PQ) bytes before classical bytes, and apply the SHA3-256 Hash-CT transcript of Equation (1). Signature wrappers implement the concat AND-combiner and serialization defined in Section 2.2, with sizes taken from the algorithm descriptors.

Post-quantum components come from KpqC/NIST source trees, pqm4 [16], PQClean [31], and public Cortex-M4 repositories. We use optimized Cortex-M4 backends where available and identify portable or reference code when it remains on the measured path. Table 1 lists the measured backend choices and provenance references.

Artifact provenance. The artifact records source snapshots for the remaining components together with build flags, generated comma-separated value files and logs, stack and object-size outputs, and the scripts used to regenerate the tables and figures.

We contribute local arithmetic only for SMAUG-T. The backend replaces the reference C polynomial-multiplication path for the NTT-unfriendly SMAUG-T ring [9] with hand-written ARMv7-M Toom-Cook/Karatsuba code, and deterministic test-vector outputs match those of the reference backend. The kernel contains no explicit secret-dependent branches or table lookups, but no formal leakage analysis is claimed. Because prior Cortex-M4 SMAUG-T work exists [23], we treat this backend only as a case study of optimization dilution within a hybrid KEM stack.

The library uses static allocation where possible. Temporary wrapper buffers have compile-time maximum sizes. Scheme-owned code size is reported as

Table 1: Measured component backends and source lineage.

Component	Measured backend	References
KEM/DSA orchestration	Hash-CT KEM wrapper and Bindel et al. DSA AND-combiner wrapper/dispatch	[2,34,4]
Harness	pqm4 hardware abstraction layer (HAL) + SysTick harness	[16]
ML-KEM	PQClean/pqm4 `m4fspeed/m4fstack`	[24,31,16]
ML-DSA	PQClean/pqm4 `m4f`	[25,31,16]
FN-DSA	mupq/pqm4 Falcon/FN-DSA line	[29,30]
SPHINCS$^+$	PQClean portable scheme code + ARMv7-M SHA-256 or pqm4 Keccak	[26,31]
HQC	v5-compatible additive fast Fourier transform (FFT) M4 backend	[27,7]
SMAUG-T	KpqClean Ver2 + local `m4opt` polymul assembly	[17,9]
NTRU+	KpqClean Ver2 + public Cortex-M4 number-theoretic transform (NTT) backend	[17,10]
HAETAE	KpqClean Ver2 + Cortex-M4 backend	[17,8]
AIMer	KpqClean Ver2 port of streaming M4 backend	[17,19]
Classical curves	Lenngren X25519/P-256 assembly, TweetNaCl Ed25519 C, and mbedTLS-C ECDSA-P256	[21,20,22,3]
Symmetric	assembly Keccak-f[1600] + ARMv7-M SHA-256	[1,33]

Source commits: X-Wing `c696a197`, pqm4 `a24bb4b6`, mupq `ddccced`, and PQClean `c3e6861`. Standalone probes check that intended backend paths are linked.

an object-level estimate that sums the PQ `.text` sections and the measured classical-leg and combiner objects, while excluding shared Keccak, HAL, and harness code. These are therefore incremental code-size estimates, not complete linked firmware images.

3.2 Target, Toolchain, and Reproducibility

Measurements run on a bare-metal ARM Cortex-M4F using the pqm4 HAL. The primary campaign uses arm-none-eabi-gcc 13.2.1, assembly (ASM) Keccak-f[1600] unless stated otherwise, and mbedTLS 3.6.3 where no dedicated Cortex-M4 classical backend is available. It runs on the NUCLEO-L4R5ZI target at 20 MHz/0 flash wait-states with release build flags `-mcpu=cortex-m4 -mthumb -mfloat-abi=hard -mfpu=fpv4-sp-d16 -O3 -fomit-frame-pointer`. Cycle counts are the primary comparison metric. Functional validation includes component known-answer tests, hybrid shared-secret agreement, and SMAUG-T reference/ASM test-vector agreement.

3.3 Keccak Implementation Variants

The Keccak experiment compares the portable C Keccak-f[1600] permutation with the pqm4 ARMv7-M backend [1]. Only the innermost permutation changes. Absorption, squeezing, padding, arithmetic kernels, classical code, target configuration, and the harness stay fixed. Ratios therefore attribute cost to the shared Keccak permutation within this stack, rather than to changes in higher-level SHAKE/SHA3 code or arithmetic kernels.

3.4 Measurement Methodology

We measure processor cycles with the pqm4 HAL. The HAL extends the 24-bit SysTick counter with overflow tracking and returns 64-bit values. Primary KEM, HAETAE, ML-DSA, and FN-DSA measurements report $N = 100$ medians. For randomized or rejection-sampling operations, Table 5 also reports means and observed maximum-to-median ratios from the measurement records. These descriptive statistics do not support estimates of latency percentiles, exceedance probabilities, or worst-case execution time.

AIMer and SPHINCS$^+$ use smaller repetition counts, as stated in the corresponding table notes. Those rows support order-of-magnitude and backend-attribution comparisons. Variability comparisons are limited to the $N = 100$ rows.

4 Bare-Metal Cortex-M4 Evaluation

This section reports NUCLEO-L4R5ZI measurements at 20 MHz/0 flash wait-states (0 WS). The Cortex-M4F target provides 640 KB static random-access memory (SRAM) and 2 MB flash. All timing values are cycle counts obtained with the backends in Table 1 unless stated otherwise.

4.1 Hybrid KEM Results

Table 2 reports the main Hash-CT X25519 benchmark configuration with $N = 100$ medians. The KeyGen column uses the independent component calls described in Section 2.1. Total denotes the summed KeyGen, Encaps, and Decaps medians, not single-party or end-to-end protocol latency. ML-KEM, NTRU+, and SMAUG-T Total values are 3,719,838–8,770,987 cycles, whereas HQC Total values are 19,637,683–74,467,370 cycles with the v5-compatible additive-FFT backend [7]. These values include ct_{pq} hashing in the benchmark transcript, so large-ciphertext rows such as HQC include an additional Hash-CT hashing cost relative to a no-ct_{pq} transcript. The results characterize this benchmark interface and do not supply a security analysis for each substituted KEM.

Standalone measurements give 0.616/1.230/0.615 M cycles for X25519 key generation, encapsulation, and decapsulation and 5.94/11.92/5.99 M cycles for Ed25519 key generation, signing, and verification. Replacing X25519 with P-256 changes all twelve KEM Total values by at most 0.347% (per-row ratios stay within 0.997–1.000×) and does not change the family ordering. Deterministic concat-to-nested checks change representative signature rows by at most 0.35% and confirm backend selection and wrapper implementation.

4.2 Hybrid DSA Results and Observed Variability

Signature measurements use the Ed25519 concat baseline unless stated otherwise. Table 3 gives the $N = 100$ HAETAE, ML-DSA, and FN-DSA rows used

Table 2: Hash-CT hybrid KEM measurements on Cortex-M4.

X25519, 20 MHz/0 WS, $N = 100$ medians. Total is the sum of the three displayed operation medians and is not protocol latency. Cycles include ct_{pq} hashing.

PQ-KEM	KeyGen	Encaps	Decaps	Total
SMAUG-T1	1,210,134	1,820,980	1,412,908	4,444,022
SMAUG-T3	1,836,280	2,439,134	2,128,255	6,403,669
SMAUG-T5	2,606,616	3,189,011	2,975,360	8,770,987
NTRU+768	1,043,984	1,646,596	1,054,290	3,744,870
NTRU+864	1,156,249	1,753,290	1,175,628	4,085,167
NTRU+1152	1,314,452	1,910,138	1,342,069	4,566,659
ML-KEM-512	1,010,490	1,643,740	1,065,608	3,719,838
ML-KEM-768	1,258,669	1,909,217	1,342,781	4,510,667
ML-KEM-1024	1,635,681	2,281,523	1,728,766	5,645,970
HQC-128	3,596,009	6,536,578	9,505,096	19,637,683
HQC-192	8,854,451	16,036,945	23,970,881	48,862,277
HQC-256	13,802,173	24,020,196	36,645,001	74,467,370

for the main comparison, while Table 4 keeps low-repetition AIMer/SPHINCS$^+$ scale-only rows separate. ML-DSA has the lowest measured signing latency among these lattice schemes, HAETAE shows rejection-sampling variability, and FN-DSA combines fast verification with expensive key generation. Because several key-generation and signing paths are randomized, Table 5 reports descriptive variability summaries for the same set instead of relying only on medians.

Table 3: Primary hybrid DSA measurements on Cortex-M4.

PQ + Ed25519, concat mode, 20 MHz/0 WS. Values are cycle-count medians over $N = 100$ runs.

PQ-DSA	KeyGen	Sign	Verify
ML-DSA-44	7,364,995	15,037,912	7,563,684
ML-DSA-65	8,460,040	17,092,257	8,559,121
ML-DSA-87	10,220,574	18,087,747	10,316,385
HAETAE2	12,028,569	31,801,353	6,943,470
HAETAE3	15,915,932	36,271,144	7,682,232
HAETAE5	19,076,721	41,939,790	8,116,242
FN-DSA-512	75,087,719	34,401,102	6,545,554
FN-DSA-1024	263,700,870	60,320,802	6,921,437

Observed signing-time variability. Across the $N = 100$ measurements, HAETAE5 has a lower signing median than FN-DSA-1024 (41.94 versus 60.32 M cycles), but its observed maximum reaches 189.8 M cycles (4.53× its median). The FN-DSA-1024 maximum is 60.58 M cycles, less than 1% above its median. HAETAE has a wider observed signing-time range than ML-DSA and FN-DSA, but these sample maxima do not determine exceedance probabilities, latency percentiles, or worst-case execution times.

Table 4: Additional low-repetition hybrid DSA measurements.

PQ + Ed25519, concat mode, 20 MHz/0 WS. AIMer192f/256s use $N = 10$, AIMer128f is a single measurement, and SPHINCS$^+$ uses $N = 5$. These rows support order-of-magnitude comparisons only.

PQ-DSA	KeyGen	Sign	Verify
AIMer128f (m4opt)	6,404,117	47,935,060	32,960,014
AIMer192f (m4fast)	7,181,956	89,527,934	78,413,565
AIMer256s (m4stack)	10,190,847	1,971,649,069	1,550,023,748
SPHINCS$^+$-SHA2-128f	20,670,562	356,674,443	27,764,750
SPHINCS$^+$-SHA2-128s	948,458,267	7,170,561,799	13,657,848
SPHINCS$^+$-SHA2-256f	64,504,322	1,309,374,104	42,139,178
SPHINCS$^+$-SHAKE-128f	56,181,392	1,188,091,067	75,088,359
SPHINCS$^+$-SHAKE-256f	204,948,947	4,015,467,171	114,616,346
SPHINCS$^+$-SHAKE-192s	4,774,330,073	42,882,388,698	41,997,450

Table 5: Observed key-generation and signing variability ($N = 100$).

Ed25519 concat mode, 20 MHz/0 WS.

Scheme	KeyGen			Sign		
	Median (M cycles)	Mean (M cycles)	Max/ median	Median (M cycles)	Mean (M cycles)	Max/ median
HAETAE2	12.03	13.81	3.12×	31.80	37.22	4.62×
HAETAE3	15.92	18.04	2.71×	36.27	41.32	3.06×
HAETAE5	19.08	23.00	5.03×	41.94	51.23	4.53×
ML-DSA-44	7.36	7.37	1.00×	15.04	15.77	1.82×
ML-DSA-65	8.46	8.46	1.00×	17.09	17.87	1.85×
ML-DSA-87	10.22	10.22	1.00×	18.09	19.38	1.86×
FN-DSA-512	75.09	81.50	2.76×	34.40	34.40	1.01×
FN-DSA-1024	263.70	313.98	3.15×	60.32	60.33	1.00×

4.3 Keccak and Hash Backend Effects

The Keccak experiment changes only Keccak-$f[1600]$. Table 6 lists representative rows from the Keccak-isolation campaign. Control rows (SPHINCS$^+$-SHA2 and FN-DSA) remain at 1.00×, supporting attribution to the Keccak permutation rather than a global build change. Speedups reach 2.16× for SPHINCS$^+$-SHAKE-256f signing and 1.65× for AIMer192s, versus 1.18–1.51× for lattice rows and 1.10× for HQC-128. This larger effect on SHAKE-heavy signatures is not unique to hybrid composition and serves here as a backend-control experiment under the common harness.

4.4 SMAUG-T Local Backend and Optimization Dilution

Table 7 isolates the local SMAUG-T arithmetic backend. Standalone gains of 1.79–1.82× shrink to 1.42–1.65× with X25519 because non-PQ work remains fixed. The 2.56 M-cycle residual is 58%, 40%, and 29% of the primary X25519 total for T1/T3/T5.

Table 6: Keccak-$f[1600]$ backend attribution.

This separate matched campaign need not reproduce the primary medians in Table 2. Speedup is the reference C cycle count divided by the ARM assembly cycle count. All non-permutation code and measurement settings are unchanged.

Class	Row	C-ref	ASM	Speedup
Lattice KEM	ML-KEM-1024 Total	8,513,176	5,645,764	1.51×
HQC	HQC-128 Total	21,575,619	19,647,266	1.10×
ML-DSA	ML-DSA-87 sign	23,042,134	18,087,747	1.27×
HAETAE	HAETAE5 sign	63,096,274	41,939,790	1.50×
AIMer	AIMer192s (m4stack) sign	1,541,036,168	934,921,177	1.65×
SPHINCS$^+$-SHAKE	SPHINCS$^+$-SHAKE-256f sign	8,663,447,709	4,015,467,171	2.16×
Control row	SPHINCS$^+$-SHA2-128f sign	356,601,179	356,674,443	1.00×
Control row	FN-DSA-512 sign	34,392,093	34,401,102	1.00×

Table 7: Standalone and composed costs of the local SMAUG-T backend.

Matched run with C Keccak. Residual is Hybrid ASM − Stand ASM in millions of cycles. Residual share is this value divided by the corresponding primary X25519 total with assembly Keccak.

Scheme	Stand C	Stand ASM	Hybrid C	Hybrid ASM	Hybrid speedup	Residual	Residual share
SMAUG-T1	5,093,516	2,811,333	7,653,239	5,371,074	1.42×	2.56	58%
SMAUG-T3	10,352,582	5,788,205	12,912,889	8,348,500	1.55×	2.56	40%
SMAUG-T5	16,834,070	9,226,662	19,395,083	11,787,708	1.65×	2.56	29%

Table 8: Wire size and implementation footprint of representative compositions.

All values are bytes. Code estimates exclude shared Keccak/HAL/harness. The final line gives representative linked-harness flash and memory.

Hybrid	Type	Public key	Ciphertext/ signature	Peak stack	Code estimate
ML-KEM-768 + X25519	KEM	1,216	1,120	6,748	20,460
SMAUG-T3 + X25519	KEM	1,120	1,024	18,336	15,988
HQC-128 + X25519	KEM	2,273	4,465	53,600	216,335
ML-DSA-44 + Ed25519	DSA	1,344	2,488	45,216	29,288
HAETAE2 + Ed25519	DSA	1,024	1,542	83,468	39,048
AIMer128f + Ed25519	DSA	64	5,956	11,952	22,820
SPHINCS$^+$-SHAKE-128f + Ed25519	DSA	64	17,156	18,860	13,880

Full linked harness flash/memory (text+data / data+bss) is 1,024,416/12,312 B for KEM rows and 643,484/23,852 B for DSA rows.

4.5 Size and Memory Considerations

Table 8 combines wire size and implementation footprint. The largest reported stack watermarks—83,468 B for HAETAE2+Ed25519 signing and 53,600 B for HQC-128+X25519 decapsulation—fit the target's 640 KB SRAM but do not directly generalize to smaller platforms.

Limitations. The campaign uses one NUCLEO-L4R5ZI board, one Cortex-M4 backend stack, and the 20 MHz/0 WS setting. It measures primitive/harness-layer timing and footprint. It does not evaluate TLS/X.509 integration, energy, resistance to side-channel or fault attacks, or worst-case execution time. The Hash-CT rows are not IETF X-Wing deployment costs. The $N = 100$ signing summaries are descriptive and do not establish protocol latency or tail probabilities.

5 Conclusion

Using one bare-metal ARM Cortex-M4 harness, we measured KpqC/NIST hybrid KEM benchmark rows with X25519 under a uniform Hash-CT transcript and Bindel et al. hybrid signature AND-combiner rows. The resulting baseline supports composed-cost attribution across the measured families.

The main lesson is that composed hybrid benchmarks behave differently from primitive-only rankings. Shared backends affect SHAKE-heavy signatures, median-only tables hide observed variability, and fixed costs from classical operations, hashing, and wrapper logic dilute standalone optimization gains. These results are presented as controls and case studies and are not claimed to be unique to hybrid composition.

Hybrid PQC reports should document backends and sample counts and evaluate optimizations within the full composition.

6 Acknowledgment

This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government(MSIT) (No.RS-2025-02306395, Development and Demonstration of PQC-Based Joint Certificate PKI Technology, 25%) and this work was supported by the Institute of Information & Communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (No. RS-2025-25394739, Development of Security Enhancement Technology for Industrial Control Systems Based on S/HBOM Supply Chain Protection, 25%) and this work was partly supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. RS-2026-25519543, Development of PQC Migration Technology for Quantum-Safe IC Chip, 25%) and this work was supported by the IITP(Institute of Information & Coummunications Technology Planning & Evaluation)-ITRC(Information Technology Research Center) grant funded by the Korea government(Ministry of Science and ICT)(IITP-2026-RS-2020-II201797, 25%).

References

1. Adomnicai, A.: An update on keccak performance on armv7-m. Cryptology ePrint Archive (2023)

2. Barbosa, M., Connolly, D., Duarte, J.D., Kaiser, A., Schwabe, P., Varner, K., Westerbaan, B.: X-wing: The hybrid kem you've been looking for. Cryptology ePrint Archive (2024)
3. Bernstein, D.J., Van Gastel, B., Janssen, W., Lange, T., Schwabe, P., Smetsers, S.: Tweetnacl: A crypto library in 100 tweets. In: International Conference on Cryptology and Information Security in Latin America. pp. 64–83. Springer (2014)
4. Bindel, N., Herath, U., McKague, M., Stebila, D.: Transitioning to a quantum-resistant public key infrastructure. In: International Workshop on Post-Quantum Cryptography. pp. 384–405. Springer (2017)
5. Bos, J.W., Costello, C., Naehrig, M., Stebila, D.: Post-quantum key exchange for the tls protocol from the ring learning with errors problem. In: 2015 IEEE symposium on security and privacy. pp. 553–570. IEEE (2015)
6. Bürstinghaus-Steinbach, K., Krauß, C., Niederhagen, R., Schneider, M.: Post-quantum tls on embedded systems: Integrating and evaluating kyber and sphincs+ with mbed tls. In: Proceedings of the 15th ACM Asia Conference on Computer and Communications Security. pp. 841–852 (2020)
7. Chen, M.S., Chien, T.Y., Chiu, C.M., Lin, H.H., Peng, C.T., Yang, B.Y.: Additive ffts for hqc on arm cortex-m4, revisited. Cryptology ePrint Archive (2026)
8. Cheon, J.H., Choe, H., Devevey, J., Güneysu, T., Hong, D., Krausz, M., Land, G., Möller, M., Stehlé, D., Yi, M.: Haetae: Shorter lattice-based fiat-shamir signatures. IACR Transactions on Cryptographic Hardware and Embedded Systems **2024**(3), 25–75 (2024)
9. Cheon, J.H., Choe, H., Seo, J., Seong, H.: Smaug (-t), revisited: Timing-secure, more compact, less failure. IEEE Access **12**, 188386–188397 (2024)
10. Cho, S.: NTRU+ KEM on ARM Cortex-M4: Optimized implementation and benchmarks. GitHub repository, https://github.com/SuBeen-Cho/Optimization_CortexM4_NTRUplus (2026)
11. Connolly, D., Hövelmanns, K., Hülsing, A., Kousidis, S., Meijers, M.: Starfighters—on the general applicability of x-wing. Cryptology ePrint Archive (2025)
12. Connolly, D., Schwabe, P., Westerbaan, B.: X-wing: General-purpose hybrid post-quantum kem. Internet-Draft draft-connolly-cfrg-xwing-kem-10, Internet Engineering Task Force (Mar 2026), https://datatracker.ietf.org/doc/draft-connolly-cfrg-xwing-kem/, version 10, 2 March 2026
13. Driscoll, F., Parsons, M., Hale, B.: Terminology for post-quantum traditional hybrid schemes. RFC 9794, IETF (2025)
14. Ghinea, D., Kaczmarczyck, F., Pullman, J., Cretin, J., Kölbl, S., Misoczki, R., Picod, J.M., Invernizzi, L., Bursztein, E.: Hybrid post-quantum signatures in hardware security keys. In: Applied Cryptography and Network Security Workshops. Lecture Notes in Computer Science, vol. 13907, pp. 480–499. Springer (2023). https://doi.org/10.1007/978-3-031-41181-6_26
15. Gonzalez, R., Wiggers, T.: Kemtls vs. post-quantum tls: Performance on embedded systems. In: International Conference on Security, Privacy, and Applied Cryptography Engineering. pp. 99–117. Springer (2022)
16. Kannwischer, M.J., Rijneveld, J., Schwabe, P., Stoffelen, K.: pqm4: Testing and benchmarking nist pqc on arm cortex-m4. NIST Second Post-Quantum Cryptography Standardization Conference (2019)
17. KpqC-cryptocraft: KpqClean Ver2: Reference implementations of the KpqC standardized algorithms. GitHub repository, https://github.com/kpqc-cryptocraft/KpqClean_ver2 (2026)

18. KpqC Organizing Committee: Selected algorithms from the KpqC competition round 2. https://kpqc.or.kr/competition_02.html (2025), final algorithms announced 16 January 2025, accessed 2026-06-26
19. Kwon, J., Lee, S., Lee, B., Seo, H., Cho, J.: Efficient implementations of aimer post-quantum signature scheme for low-end to high-end iot devices. IEEE Internet of Things Journal (2025)
20. Lenngren, E.: P256-Cortex-M4: P-256 ECDSA/ECDH for Cortex-M4. https://github.com/Emill/P256-Cortex-M4
21. Lenngren, E.: X25519-Cortex-M4. https://github.com/Emill/X25519-Cortex-M4
22. Mbed TLS Team: Mbed TLS. GitHub repository, https://github.com/Mbed-TLS/mbedtls (2024)
23. Narlı, O., Pehlivanoğlu, M.K., Akleylek, S.: A new optimized implementation of smaug-t for lightweight devices. In: Nordic Conference on Secure IT Systems. pp. 77–90. Springer (2025)
24. National Institute of Standards and Technology: FIPS 203: Module-lattice-based key-encapsulation mechanism standard (ML-KEM). Federal information processing standards publication, NIST (Aug 2024)
25. National Institute of Standards and Technology: FIPS 204: Module-lattice-based digital signature standard (ML-DSA). Federal information processing standards publication, NIST (Aug 2024)
26. National Institute of Standards and Technology: FIPS 205: Stateless hash-based digital signature standard (SLH-DSA). Federal information processing standards publication, NIST (Aug 2024)
27. National Institute of Standards and Technology: NIST IR 8545: Status report on the fourth round of the NIST post-quantum cryptography standardization process. Tech. rep., NIST (Mar 2025), HQC selected for standardization
28. National Institute of Standards and Technology: Recommendations for key-encapsulation mechanisms. NIST Special Publication 800-227, NIST (Sep 2025). https://doi.org/10.6028/NIST.SP.800-227
29. Perlner, R.: FIPS 206 status update: FN-DSA (Falcon). Presentation, 6th NIST PQC Standardization Conference (2025), nIST
30. Pornin, T.: Falcon on ARM Cortex-M4: An update. Cryptology ePrint Archive, Report 2025/123 (2025)
31. PQClean Contributors: PQClean: Clean, portable, tested implementations of post-quantum cryptography. https://github.com/PQClean/PQClean
32. Sim, M., Song, G., Eum, S., Lee, M., Yoon, S., Baksi, A., Seo, H.: Integrating and benchmarking kpqc in tls/x. 509. Electronics **14**(18), 3717 (2025)
33. Weatherley, R.: ARMv7-M SHA-256 assembly. GitHub repository, https://github.com/rweather/lwc-finalists
34. X-Wing KEM Team: X-Wing C implementation. GitHub repository, https://github.com/x-wing-kem-team/xwing (2024), commit `c696a197`, accessed 2026-06-27